

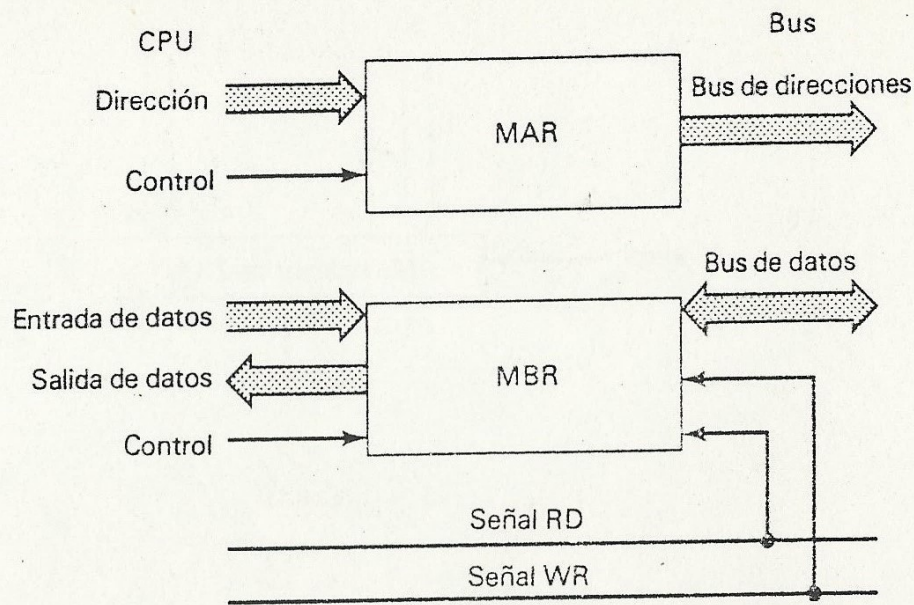


Fig. 4-6. Dos registros que se utilizan para los buses de direcciones y datos.

de direcciones del sistema siempre están permitidas [o, posiblemente, sólo durante las lecturas y las escrituras, lo que requiere una línea de permiso de salida conectada al OR lógico de RD y WR (no mostrada en la figura)]. La línea de control del MBR hace que los datos se carguen del bus "Entrada de datos", en el lado de la CPU. El bus "Salida de datos" está siempre permitido.

El bus de datos del sistema es bidireccional, sacando el contenido del MBR cuando WR está activa y cargándolo cuando se activa RD.

4.1.7. Encapsulado de los componentes

En las secciones precedentes hemos descrito varios circuitos que pueden combinarse para formar una computadora. Se hallan disponibles en el comercio en formas conceptualmente distintas. La más simple es la combinación en circuitos MSI (Integrados a Escala Media), con componentes como un registro, una ALU, un registro de corrimiento, etc., cada uno en una pastilla. Este método se encuentra en la figura 4-7(a). Debido a la gran variedad de chips rápidos y de bajo precio disponibles en el mercado, muchas computadoras se construyen conectando este tipo de componentes.

El principal inconveniente de construir computadoras con componentes MSI es el gran número de componentes necesarios, que ocupan numerosas tarjetas de circuito impreso, consumen mucha energía y disipan abundante calor. Otra técnica es usar pastillas de **rebanada de 1 bit**. Cada pastilla de rodaja de bit tiene, por ejemplo, 1 bit de registros, ALU y otros componentes. La figura 3-20 muestra el aspecto interno que presenta una rodaja de ALU de 1 bit. Podríamos fácilmente extender su diseño para añadirle, por ejemplo, 16 registros de 1 bit, un registro de corrimiento de 1 bit y otros componentes de 1 bit de ancho.

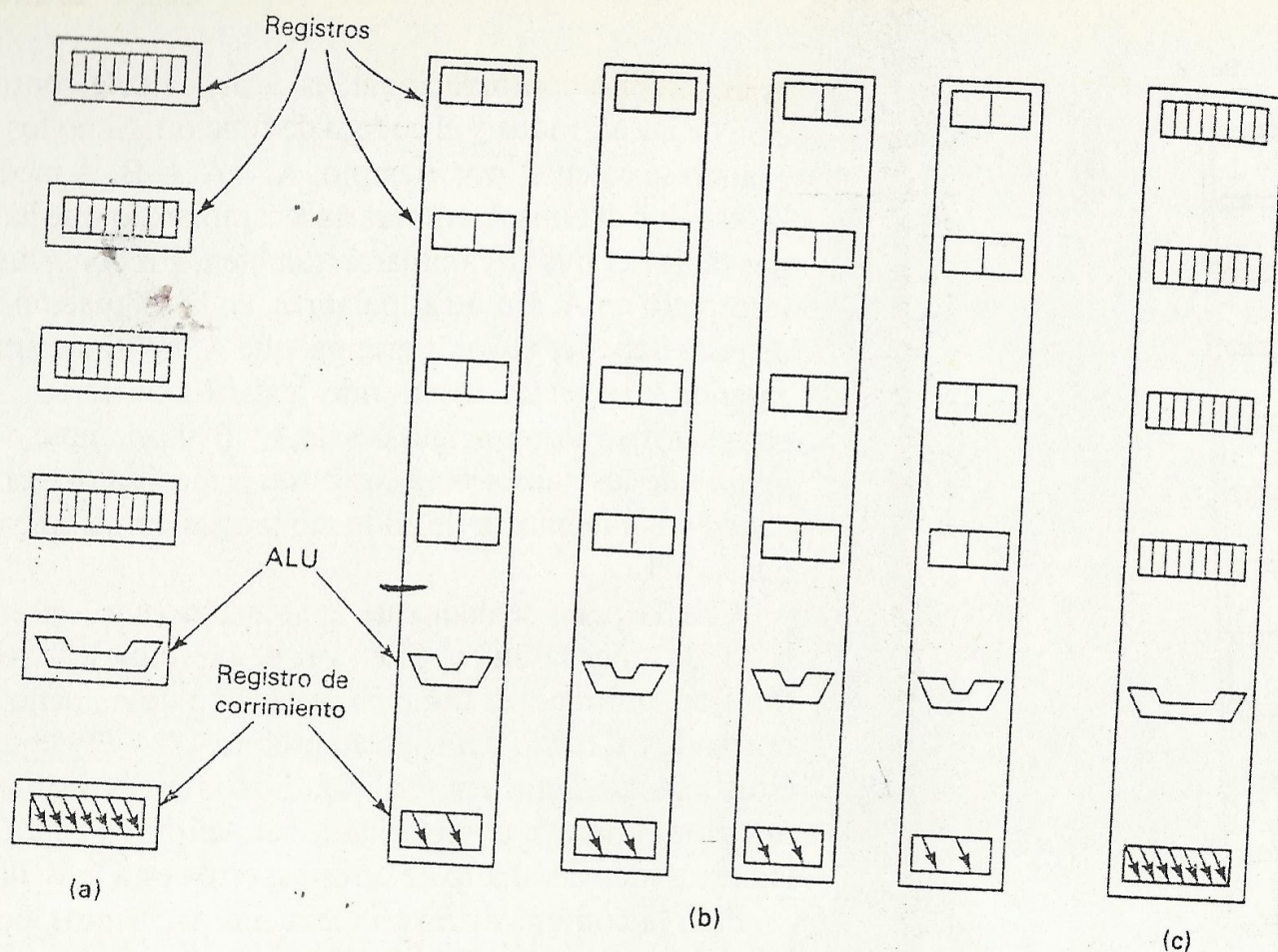


Fig. 4-7. Tres modos de construir una computadora. (a) Pastillas MSI. (b) Rebanadas de bit. (c) Pastilla LSI.

Tomando, por ejemplo, 32 de esas pastillas y poniéndolas una al lado de otra, obtendríamos una máquina con 32 registros, una ALU, un registro de corrimiento, etc., todos de 32 bits.

En esta forma podríamos construir una máquina de 16 bits con solo 16 pastillas. Las rebanadas de bit dan al diseñador la posibilidad de construir fácilmente una máquina de cualquier longitud de palabra. También hay pastillas con rebanadas de 2 o 4 bits. La figura 4.7(b) muestra una máquina de 8 bits construida con 4 rebanadas de 2 bits. En general, la utilización de rebanadas requiere menos pastillas y mucho menos tiempo de diseño que la de circuito MSI, pero normalmente produce máquinas más lentas.

Una tercera forma de combinar los componentes consiste en poner el procesador completo en una pastilla [véase la figura 4-7(c)]. A pesar de que reduce de modo extraordinario el número de pastillas (a una sola), adolece de desventajas. Primero, la tecnología necesaria para integrar un gran número de componentes en una pastilla es diferente de la utilizada en los circuitos integrados de tipo MSI o en los de rebanada de 1 bit y suele producir máquinas más lentas. Además, las técnicas de diseño y fabricación son extraordinariamente complicadas

y costosas. Por el contrario, cualquier competente ingeniero en electrónica puede diseñar una computadora sencilla con pastillas MSI o de rebanada de 1 bit sin demasiados problemas. Desde el punto de vista del fabricante, que espera construir computadoras durante años, quizá merezca la pena dominar las tecnologías necesarias para hacer procesadores en una pastilla. Sin embargo, seguramente no le interesará a una empresa que necesite sólo una máquina de propósito específico. Así pues, las opciones son: utilizar un procesador disponible en el comercio, contratar una compañía especializada en el diseño y fabricación de una pastilla de propósito específico, o construir el procesador con componentes MSI o de rodajas de bit.

4.2. UNA MICROARQUITECTURA TIPICA

Una vez estudiados todos los componentes básicos con los que se construye el nivel de microprogramación, es el momento de ver cómo se interconectan. Como los principios generales en esta área son pocos y dispersos, abordaremos el tema por medio de un ejemplo detallado.

4.2.1. La ruta de datos

La **ruta de datos** es la parte de la CPU que contiene a la ALU, sus entradas y sus salidas. La de nuestro ejemplo se muestra en la figura 4-8. Contiene 16 registros idénticos de 16 bits, rotulados PC, AC, SP, etc., que forman una memoria de anotaciones accesibles solamente al nivel de microprogramación. Los registros rotulados 0, + 1 y -1 se usarán para guardar las constantes indicadas. En nuestros ejemplos sencillos el 0 nunca se usa, pero probablemente se necesitará en una máquina más complicada; de todos modos los hemos incluido porque disponíamos de más registros que los que podíamos utilizar. Los restantes tienen nombre que explicaremos más adelante. Cada registro puede sacar su contenido a uno de los buses A o B o a ambos y tomarlo de un tercero, C, como se muestra en la figura.

Los buses A y B alimentan una ALU de 16 bits que puede realizar cuatro funciones: $A + B$, $A \text{ AND } B$, $A \text{ y NOT } A$. Las dos líneas de control de la ALU, F_0 y F_1 , especifican la función que se va a realizar. La ALU genera dos bits de estado derivados de su salida: N, que se pone a 1 cuando la salida de la ALU es negativa, y Z, que se pone a 1 cuando dicha salida es cero.

La salida de la ALU pasa por un registro de corrimiento que puede desplazarla un bit a la izquierda o a la derecha, o bien no realizar ningún desplazamiento. Es posible desplazar un registro, R, 2 bits a la izquierda, calculando $R + R$ en la ALU y desplazando la suma un bit más en el registro de corrimiento.

Ni el bus A ni el B están conectados directamente a ALU, sino que hay un par de registros buffer (tampón) intermedios. Estos se necesitan porque la ALU

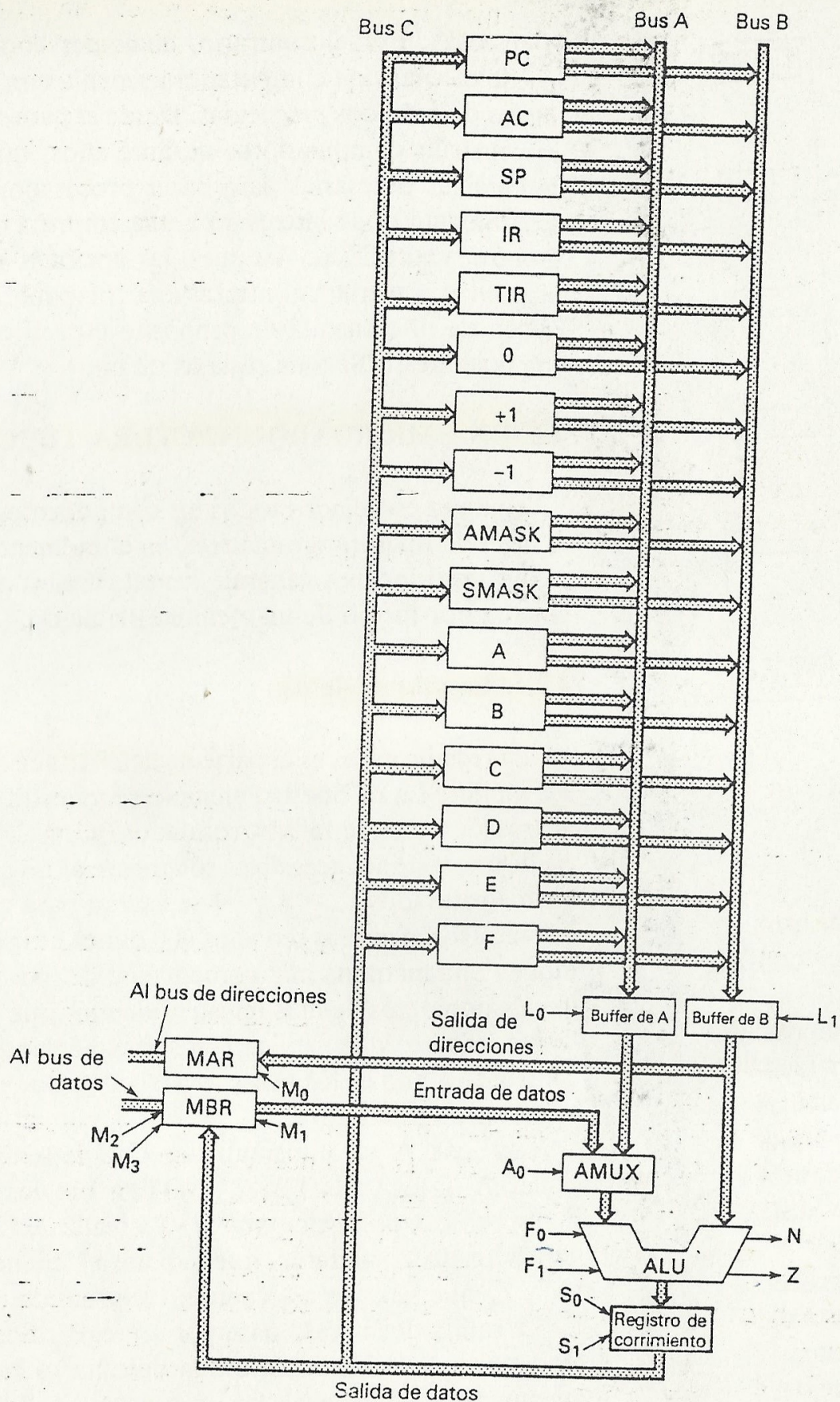


Fig. 4-8. Ruta de datos del ejemplo de microarquitectura de este capítulo.

es un circuito combinacional, es decir, calcula continuamente la salida en función de las entradas y el código de función. Si no los hubiera, habría problemas cuando se calcule, por ejemplo, $A := A + B$. A medida que A fuera cargándose, el valor del bus A empezaría a cambiar, lo que haría que la salida de ALU y, por tanto, el bus C cambiaran también. En consecuencia se guardaría un valor incorrecto en A. En otras palabras, en la asignación $A := A + B$, el A del lado derecho debe ser el valor original que A, no una extraña mezcla de bits del valor anterior y reciente. Insertando registros buffer en los buses A y B, podemos congelar los valores originales de A y B al principio del ciclo, de modo que ALU se aisle de los cambios en los buses a medida que el nuevo valor se vaya guardando en la memoria de anotaciones. La carga de estos buffers está controlada por L_0 y L_1 .

Vale la pena señalar que la solución (v.g., insertar biestables al frente de la ALU) no es la única. Si en vez de biestables, todos los registros son del tipo flip-flop, entonces es también posible un desempeño de dos buses, cargando los operandos al inicio del ciclo en los buses A y B y leyendo el resultado en uno de éstos más adelante en el ciclo. Los intercambios entre los diseños de dos y tres buses abarcan complejidad, paralelismo y cantidad de alambrado. Un tratamiento más detallado de estos aspectos está más allá del alcance de este libro.

Para la comunicación con la memoria, hemos incluido en la microarquitectura un MAR y un MBR. El MAR puede cargarse a partir del registro de B, en paralelo con una operación de la ALU. La línea M_0 controla la carga del MAR. En las escrituras se puede cargar el MBR con la salida del registro de corrimiento, en paralelo con el almacenamiento en la memoria de anotaciones o en lugar de él M_1 controla la carga del MBR a partir de la salida del registro de corrimiento M_2 y M_3 controlan las lecturas y escrituras de memoria. En las primeras, los datos leídos de la memoria pueden presentarse en la entrada izquierda de la ALU a través del multiplexor A, representado en la figura 4-8 por Amux. La línea de control A_0 determina si el biestable A o el MBR se alimentan en la ALU. La microarquitectura de la figura 4-8 es similar a la de muchos de los circuitos comerciales de rebanadas de 1 bit disponibles.

4.2.2. Microinstrucciones

Para controlar la trayectoria de datos de la figura 4-8 se requiere de 61 señales. De acuerdo a sus funciones, éstas se pueden dividir en nueve grupos que a continuación se describen.

16 para controlar la carga del bus A a partir de registros internos.

16 para controlar la carga del bus B a partir de registros internos.

16 para controlar la carga de la memoria de anotaciones a partir del bus C.

- 2 para controlar los registros de A y B. L_0 y L_1
- 2 para controlar la función de la ALU. F_0 y F_1
- 2 para controlar al registro de corrimiento. S_0 y S_1
- 4 para controlar el MAR y el MBR. M_0, M_1, M_2, M_3
- 2 para indicar una lectura o una escritura en memoria.
- 1 para controlar el Amux. A_0

Con los valores de las 61 señales podemos realizar un ciclo de nuestra ruta de datos. Un ciclo consiste en vaciar los valores en los buses A y B, almacenarlos temporalmente en los registros de A y B, en pasarlos a través de la ALU y el registro de corrimientos y en almacenar el resultado en la memoria interna y en el MBR o en ambos. Además, puede cargarse el MAR e iniciarse un ciclo de memoria. Como primera aproximación, podríamos tener un registro de control de 61 bits, cada uno de ellos, para una señal de control. Un bit a 1 significa que la señal está activada y uno a 0 que no lo está.

Sin embargo, podemos reducir en mucho el número de bits que necesitan para controlar la ruta de datos, al precio de un pequeño incremento en la circuitería. Para empezar, tenemos 16 bits para controlar la entrada al bus A, lo que permite 2^{16} combinaciones de los registros. Desafortunadamente, sólo están permitidas 16: cada uno de los 16 registros, excluyendo los demás. Por tanto, podemos codificar la información del bus A en 4 bits y utilizar un decodificador para generar las 16 señales de control. Lo mismo vale para el bus B.

Esta situación es algo diferente en el bus C. En principio, es posible tener almacenamientos múltiples en la memoria de anotaciones, pero esta práctica es casi siempre inútil y la mayoría de las máquinas no la permiten. Así pues, también podremos codificar el control del bus C con 4 bits. Habiendo ahorrado $3 \times 12 = 36$ bits, necesitamos ahora sólo 25 bits de control para activar la ruta de datos, L_0 y L_1 siempre se necesitan en un momento determinado del ciclo de máquina, por lo que puede activarlos el reloj, quedando 23 bits de control. Una señal adicional, no estrictamente necesaria pero a menudo útil, es aquella que permite o inhibe el almacenamiento del bus C en la memoria interna. En algunas situaciones sólo interesa generar las señales N y Z, pero no se desea almacenar el resultado. Con este bit adicional que llamaremos ENC (habilita C), podemos indicar que se almacene el bus C ($ENC = 1$) o no ($ENC = 0$).

En este momento estamos en condiciones de controlar la ruta de datos con 24 bits. Notamos ahora que RD puede usarse para cargar el MBR con el bus de datos del sistema y WR para permitir la salida del MBR al mismo bus. Esta observación reduce a 22 el número de señales de control independientes.

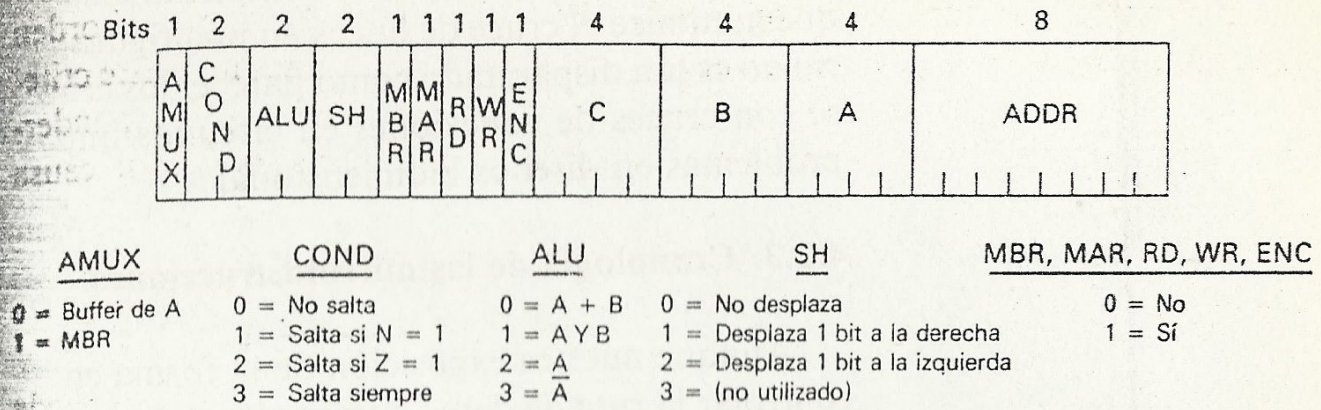


Fig. 4-9. Estructura de una microinstrucción para controlar la ruta de datos en la figura 4-8.

El siguiente paso en el diseño de la microarquitectura es inventar un formato de microinstrucción que contenga 22 bits. La figura 4-9 muestra uno de los posibles, con dos campos adicionales, COND y ADDR, que se describirán en seguida. La microinstrucción contiene 13 campos, 11 de los cuales son:

AMUX— Controla la entrada izquierda a la ALU: 0 = tampón de A, 1 = MBR.

ALU — Función de la ALU: 0 = A + B, 1 = A AND B, 2 = A, 3 = $\overline{A}$.

SH — Función del registro de corrimiento: 0 = nada, 1 = a la derecha, 2 = a la izquierda.

MBR — Carga del MBR a partir del registro de corrimiento: 0 = no carga, 1 = carga.

MAR — Carga del MAR a partir del registro de B: 0 = no carga, 1 = carga.

RD — Petición de lectura de memoria: 0 = no pide, 1 = carga el MBR de memoria.

WR — Petición de escritura en memoria: 0 = no pide, 1 = escribe el MBR en memoria.

ENC — Control de almacenamiento en registros de memoria interna: 0 = no almacena, 1 = almacena.

C — Selecciona el registro dónde almacenar si ENC = 1: 0 = PC, 1 = AC, etc.

B — Selecciona la fuente del bus B: 0 = PC, 1 = AC, etc.

A — Selecciona la fuente del bus A: 0 = PC, 1 = AC, etc.

El orden de los campos es completamente arbitrario. Se ha elegido el orden que minimice el cruce de líneas en una figura que hay más adelante. (Este criterio no es tan disparatado como parece; los cruces de líneas suelen corresponderse con cruces de conexiones en circuitos impresos o integrados, lo que causa problemas en diseños bidimensionales.)

4.2.3. Cronología de las microinstrucciones

Aunque nuestra exposición de la forma en que una microinstrucción puede controlar la ruta de datos durante un ciclo es casi completa, hemos olvidado un asunto importante: la cronología. Un ciclo básico de ALU consiste en cargar los registros A y B, darle tiempo a la ALU para realizar su trabajo y almacenar entonces el resultado. Es evidente que estos sucesos deben tener lugar en esa secuencia. Si intentáramos almacenar el bus C en la memoria de anotaciones antes de cargar los registros A y B, se guardaría basura en lugar de datos útiles. Para obtener la secuencia correcta, introducimos un reloj de cuatro fases, es decir, con cuatro subciclos, como el de la figura 4-5. He aquí los eventos clave durante cada uno de los subciclos:

1. Carga la siguiente microinstrucción a ejecutarse en un registro denominado **MIR** (Registro de MicroInstrucción).
2. Salida del contenido de los registros a los buses A y B, y su captura por los registros A y B.
3. Cuando las entradas de la ALU están estabilizadas, hay que dar tiempo a la ALU y al registro de corrimiento para que produzcan una salida estable y cargar el MAR si es necesario.
4. Ahora que la salida del registro de corrimiento está estabilizada, se almacena el bus C en la memoria de anotaciones y en el MBR si es necesario.

La figura 4-10 es un diagrama de bloques detallado de la microarquitectura de nuestra máquina completa. Puede parecer impresionante a primera vista, pero merece la pena estudiarla en detalle. Cuando el lector entienda completamente cada bloque y cada línea, podrá avanzar en la comprensión total del nivel de microprogramación. El diagrama de bloques tiene dos partes: la ruta de datos, a la izquierda, que ya estudiamos de modo pormenorizado, y la sección de control, de la que trataremos ahora.

La parte más voluminosa e importante de la porción de control de la máquina es la **memoria de control**. Es donde se guardan las microinstrucciones. En algunas máquinas es una memoria de lectura solamente, en otras es de lectura y